

Infraestructura como Código para Sistemas de IA

Módulo 9 · Infraestructura y Despliegue

Terraform, Helm, Docker, GitOps con ArgoCD, infraestructura efímera para experimentación y best practices de IaC.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Infraestructura como Código para Sistemas de IA

La Infraestructura como Código (IaC) es la práctica de gestionar y aprovisionar infraestructura de cómputo a través de código en lugar de procesos manuales. En el contexto de sistemas de IA y LLMs, IaC es especialmente crítica: la infraestructura de un sistema LLM (GPUs, redes, almacenamiento, Kubernetes clusters) es compleja, cara de configurar manualmente, y propensa a errores si cada despliegue se hace a mano. Un cluster GPU mal configurado puede producir costes de miles de dólares por día sin generar valor.

Los tres pilares de IaC para sistemas de IA son: reproducibilidad (el mismo código produce siempre la misma infraestructura), versionado (la infraestructura está en Git y cada cambio tiene un commit asociado), y automatización (el aprovisionamiento, actualización, y destrucción de infraestructura es automático y parte del pipeline CI/CD). En 2025, las herramientas estándar para IaC de sistemas de IA son: Terraform para la infraestructura cloud, Helm para la gestión de aplicaciones en Kubernetes, y Ansible para la configuración de sistemas.

Las herramientas de IaC para sistemas de IA

Terraform: aprovisionar infraestructura cloud

Terraform (HashiCorp) permite definir infraestructura cloud en archivos HCL (HashiCorp Configuration Language): clusters de Kubernetes, instancias de GPU, bases de datos vectoriales, redes, y cualquier otro recurso cloud. La ejecución de terraform plan muestra los cambios que se harán antes de aplicarlos (preview), terraform apply los ejecuta, y terraform destroy los elimina. El estado de la infraestructura se almacena en un backend (S3, GCS, Terraform Cloud) para que múltiples miembros del equipo puedan trabajar en la misma infraestructura. Para sistemas de IA en AWS, los recursos más comunes son: EKS (Kubernetes), instancias P4/P5 (GPU), S3 (almacenamiento de modelos), y RDS (base de datos).

Helm: gestionar aplicaciones Kubernetes

Helm es el gestor de paquetes de Kubernetes. Un Helm chart es un conjunto de templates YAML de Kubernetes parametrizados que definen cómo desplegar una aplicación. Para sistemas LLM, los Helm charts encapsulan: el Deployment de vLLM (con la configuración de GPU, el modelo a cargar, y los parámetros de inferencia), los Services y Ingress, los ConfigMaps con la configuración del sistema, el HPA/KEDA Autoscaler, y los PersistentVolumeClaims para el almacenamiento del modelo. El proyecto llm-d proporciona Helm charts de referencia para despliegue de vLLM con todas las mejores prácticas.

Docker: contenedores reproducibles

Todos los componentes de un sistema LLM deben estar contenedorizados con Docker para garantizar reproducibilidad. Las mejores prácticas para imágenes Docker de sistemas LLM: usar imágenes base oficiales de CUDA de NVIDIA con versión específica (no "latest"), pin de

versiones de todas las dependencias Python (requirements.txt con versiones exactas), separar el código de los pesos del modelo (el código está en la imagen, los pesos se montan desde almacenamiento externo para mantener la imagen ligera), y escaneo de la imagen con Trivy para vulnerabilidades antes del despliegue.

SECCIÓN 3 · CÓMO FUNCIONA

Un pipeline de IaC completo para sistemas LLM

El pipeline de IaC completo para un sistema LLM en producción: Terraform aprovisiona el cluster EKS/GKE con nodos GPU, la VPC y la configuración de red, el bucket S3/GCS para los pesos del modelo, y la base de datos vectorial (si se usa uno managed). Helm despliega vLLM con el Helm chart configurado para el modelo específico y los parámetros de inferencia. ArgoCD sincroniza el estado del cluster con el repositorio Git. El pipeline de CI/CD (GitHub Actions) ejecuta terraform plan en cada PR que cambia la infraestructura, y terraform apply automáticamente cuando el PR se mergea a main.

Para la gestión del estado de Terraform en un equipo: usa Terraform Cloud o un backend S3 con DynamoDB para el state locking. El state locking garantiza que solo un usuario aplica cambios a la vez, evitando conflictos cuando múltiples ingenieros trabajan en la misma infraestructura. Los workspaces de Terraform permiten gestionar múltiples entornos (dev, staging, prod) con el mismo código pero estados separados.

Infraestructura efímera para experimentación

Una práctica muy efectiva para equipos de ML es la infraestructura efímera para experimentación: los científicos de datos pueden aprovisionar un cluster GPU temporal para un experimento de fine-tuning, usarlo durante horas o días, y destruirlo automáticamente al terminar. Con Terraform, este proceso es: terraform workspace new experimento-fine-tuning-llama3, terraform apply (aprovisiona el cluster en minutos), ejecutar el fine-tuning, terraform destroy (elimina todos los recursos). El coste es solo por el tiempo de uso real, sin pagar por infraestructura ociosa.

SECCIÓN 4 · EJEMPLOS REALES

- Terraform + EKS + llm-d en empresa de SaaS IA: el equipo de plataforma gestiona un cluster EKS con 10 nodos A100 usando Terraform. Cada cambio de infraestructura pasa por un PR con terraform plan en CI. ArgoCD sincroniza los Helm charts de vLLM con el cluster. Los científicos de datos modifican solo el values.yaml del Helm chart para cambiar el modelo o la configuración de inferencia, sin tocar la infraestructura de Kubernetes directamente. El cluster completo se puede destruir y recrear en 20 minutos desde cero usando Terraform.
- Infraestructura efímera para fine-tuning (equipo de investigación): los investigadores tienen acceso a un módulo Terraform que aprovisiona un cluster GPU temporal (4x A100 en CoreWeave) con un script. Provisionamiento en 5 minutos, acceso SSH, fine-tuning de 6 horas, destrucción automática en la hora siguiente a la finalización del training job. Coste: $4 \times \$2.21 \times 7 \text{ horas} = \61.88 vs pagar por el cluster 24/7 ($\$2.21 \times 4 \times 24 \times 30 = \$6.365/\text{mes}$).
- GitOps con ArgoCD para asistente empresarial: cualquier cambio al sistema (nueva versión de vLLM, cambio de modelo, actualización del system prompt) se hace a través de

un PR a GitHub. La revisión del PR incluye la evaluación automática sobre el golden dataset. Si el PR pasa CI, ArgoCD lo despliega automáticamente al cluster de staging. Después de 24 horas de monitorización en staging, el PR se mergea a main y ArgoCD lo despliega a producción. Cero despliegues manuales.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No versionar el estado de Terraform. Sin un backend remoto para el estado de Terraform, cada ingeniero tiene su propio estado local. Esto produce conflictos cuando múltiples personas trabajan en la misma infraestructura y hace imposible saber cuál es el estado real del sistema. Siempre usa un backend remoto (S3 + DynamoDB o Terraform Cloud) desde el primer día.

Error 2: Hardcodear secretos en Terraform o en Dockerfiles. Las API keys, contraseñas de base de datos, y otros secretos nunca deben estar en el código de IaC ni en las imágenes Docker. Usa Terraform para referenciar secretos desde AWS Secrets Manager o HashiCorp Vault, y kubernetes Secrets para inyectarlos en los pods.

Error 3: No usar el comando terraform plan antes de apply en producción. terraform apply sin plan previo en producción puede producir cambios inesperados que eliminan recursos, cambian configuraciones críticas, o interrumpen el servicio. El proceso correcto es siempre: terraform plan > revisar los cambios > terraform apply con confirmación explícita. En CI/CD, el plan se ejecuta en cada PR y el apply solo en la rama main protegida.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El layout de repositorio para IaC de un sistema LLM: /infrastructure/terraform/ (módulos Terraform para el cluster GPU, la red, el almacenamiento), /infrastructure/kubernetes/ (manifiestos Kubernetes o Helm charts para vLLM, monitoring, etc.), /infrastructure/docker/ (Dockerfiles para los componentes del sistema), /infrastructure/argocd/ (configuración de ArgoCD para GitOps). Cada directorio tiene su propio pipeline de CI que valida los cambios (terraform validate, helm lint, docker build) antes del merge.

Para empezar con Terraform en AWS para un cluster EKS GPU: usa el módulo terraform-aws-modules/eks/aws (el módulo oficial de la comunidad Terraform para EKS). Define el node group con instancias GPU (instance_type = "p3.2xlarge" para una V100, "p4d.24xlarge" para 8x A100). Configura el eks-gpu-device-plugin para que los pods puedan usar GPUs. El Helm chart de vLLM se despliega sobre el cluster EKS una vez que está provisionado.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T9 (CI/CD para IA): el pipeline de CI/CD incluye los pasos de terraform plan y el despliegue Helm.
- M9-T2 (Arquitecturas de despliegue): la arquitectura de despliegue se implementa con los Helm charts y Terraform.
- M9-T4 (Seguridad): IaC incluye la configuración de seguridad de la infraestructura (Network Policies, Secrets, RBAC).

IaC para sistemas de IA usa tres herramientas: Terraform (aprovisionar infraestructura cloud: EKS, GPUs, almacenamiento), Helm (gestionar aplicaciones Kubernetes: vLLM, monitoring, ingress), y Docker (contenedores reproducibles con versiones pinned). El pipeline de IaC: Terraform plan en CI para cada PR → Terraform apply automático al merge a main → ArgoCD sincroniza Kubernetes con el estado en Git. Reglas: backend remoto de Terraform siempre (S3+DynamoDB), nunca secretos en código (usar Vault o Kubernetes Secrets), siempre plan antes de apply en producción. La infraestructura efímera (terraform destroy automático) es la mejor práctica para experimentación: paga solo por el tiempo de uso.